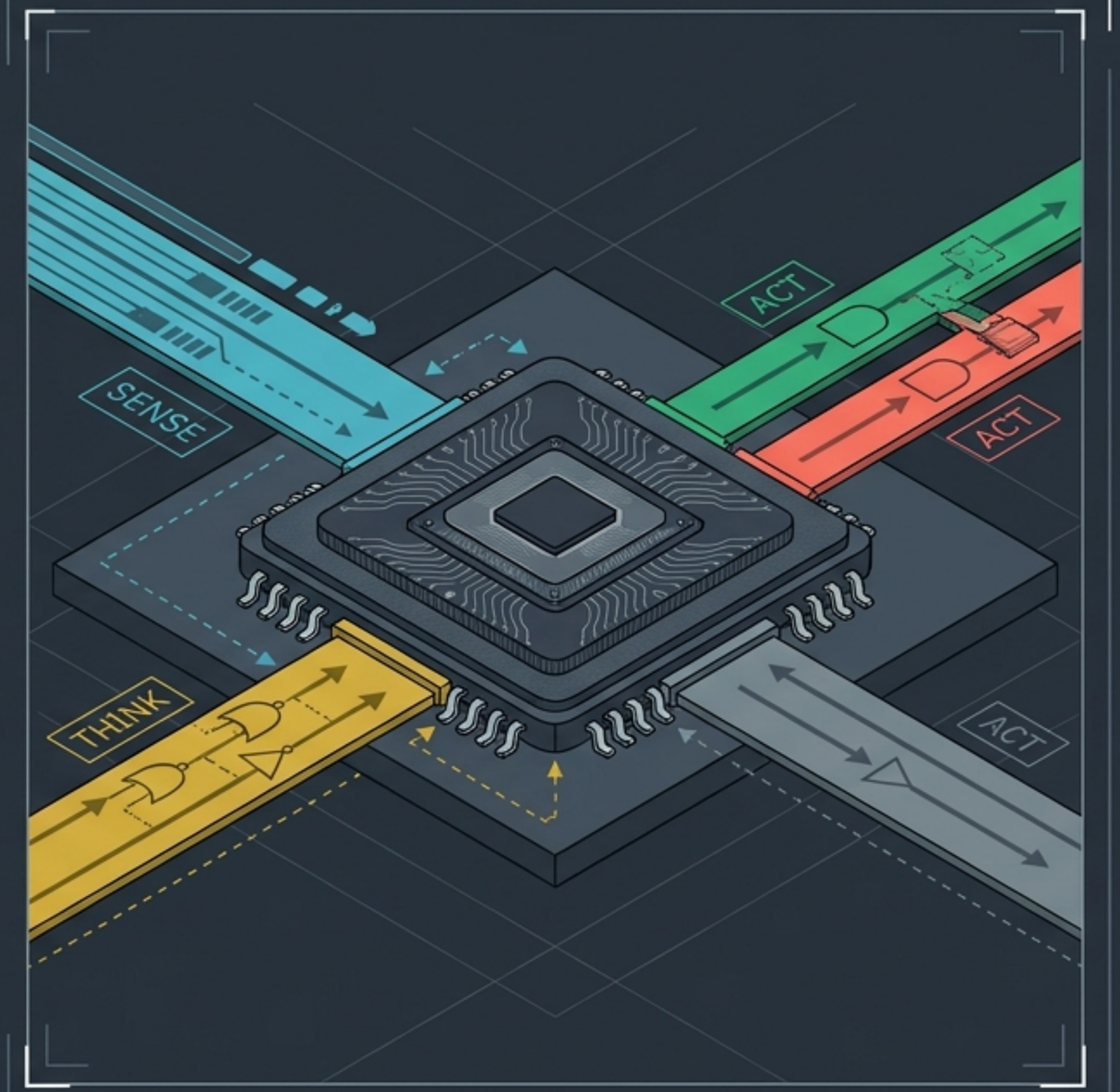
Алгоритмы работы робототехнических робототехнических средств: От команд к поведению

Очный практикум по программированию микроконтроллеров (Урок 3.3)



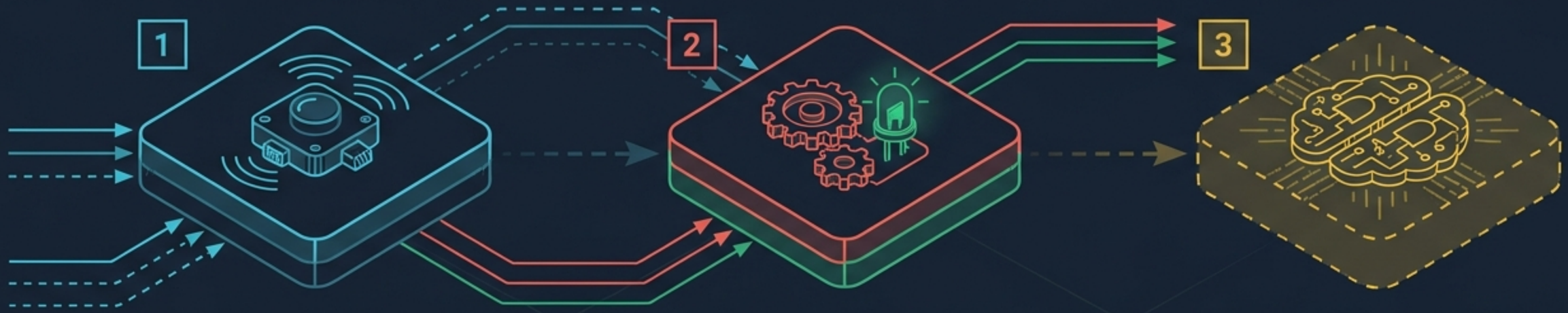
NotebookLM

Эволюция робота: Чего не хватает для автономности?

Урок 3.1: Чувства (Датчики)

Урок 3.2: Мышцы (Исполнительные устройства)

Урок 3.3: Мозг (Алгоритмы)



Считываем данные об окружающем мире.

Включаем свет, звук, движение.

Как данные превращаются в решение?

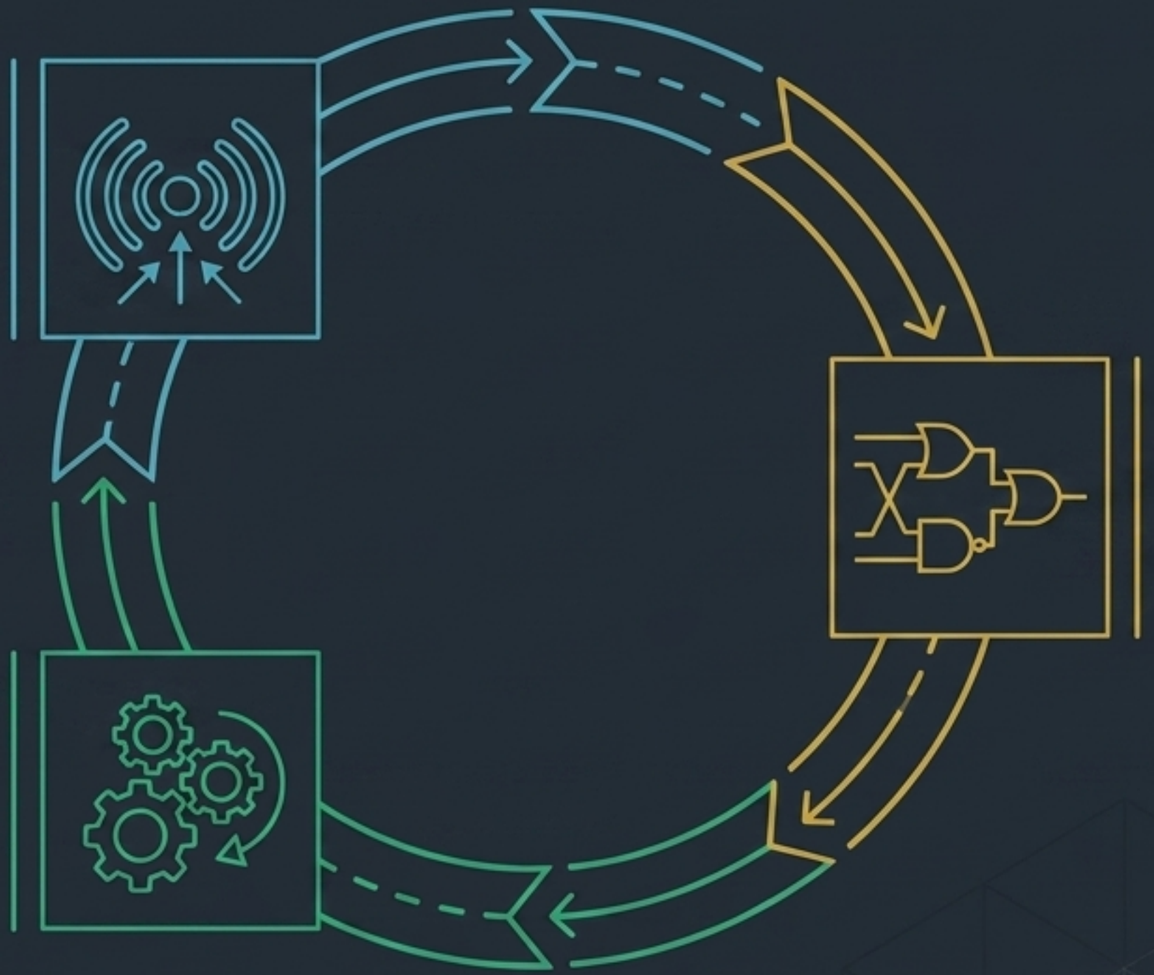
До сих пор мы передавали отдельные команды. Сегодня мы созідадим систему, способную действовать самостоятельно.

NotebookLM

Базовый цикл автономного управления

SENSE (Считать данные)

Измерить расстояние.



THINK (Оценить и выбрать)

Сравнить с порогами безопасности.

ACT (Выполнить)

Изменить индикацию или повернуть сервопривод.

Робот работает не по набору разрозненных команд, а по непрерывному алгоритму: получает данные, принимает решение, выполняет действие и повторяет цикл.

NotebookLM

Настольная модель: Имитация физического движения

Команда мобильного робота (Цель)	Наша настольная модель (Прокси)
Движение вперёд (Forward)	Зелёный LED + Монитор порта: FORWARD + Fan Motor
Предупреждение о препятствии (Warning)	Жёлтый LED
Экстренная остановка (Stop)	Красный LED + Buzzer (Звук)
Поворот (Turn)	Поворот Servo + Монитор порта: TURN LEFT / RIGHT

У нас нет колёсного шасси. Мы строим логику мозга, заменяя физическое перемещение световой, звуковой и механической индикацией состояний.

NotebookLM

Концепция 1: Пространственные зоны (Объезд препятствий)

Зеленая зона (> 30 см): SAFE.
Действие: Двигаться вперёд.

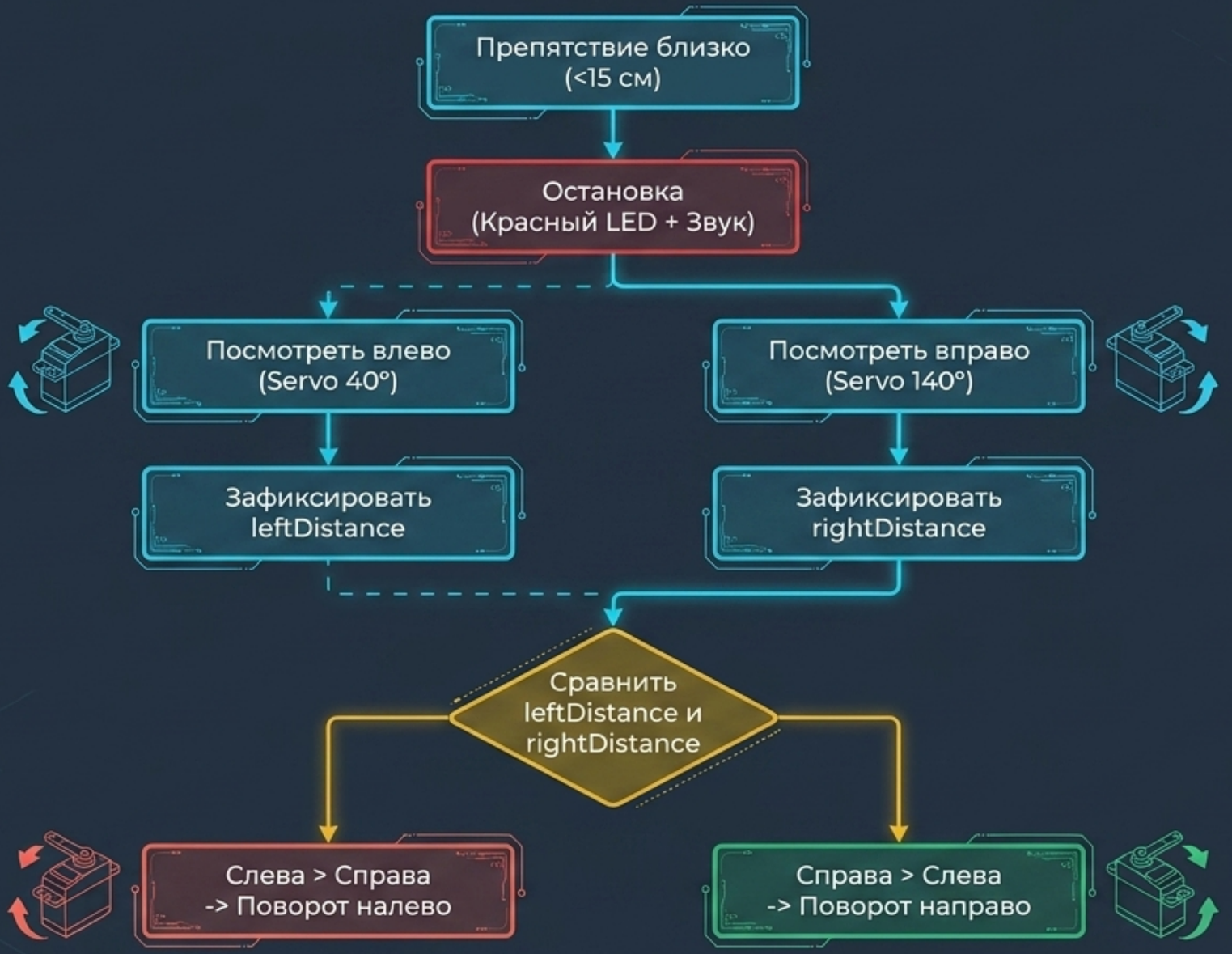
Желтая зона (15 - 30 см): WARNING.
Действие: Снизить скорость / Предупредить.

Красная зона (< 15 см): STOP.
Действие: Остановка и поиск пути.

Алгоритм делит непрерывный поток данных о расстоянии на три дискретные зоны принятия решений.

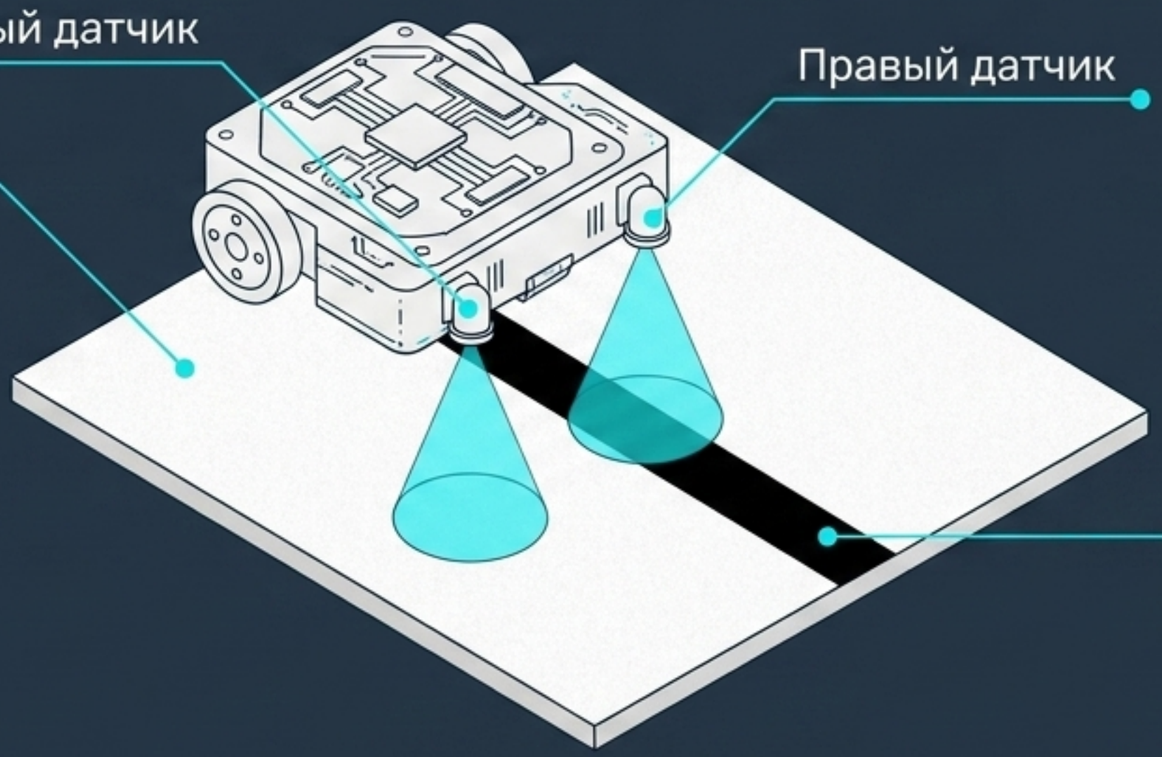
NotebookLM

Алгоритм сканирования в зоне STOP



NotebookLM

Концепция 2: Идея движения по линии



Цель алгоритма: удерживать оба датчика на границе линии, постоянно корректируя курс при съезде.

NotebookLM

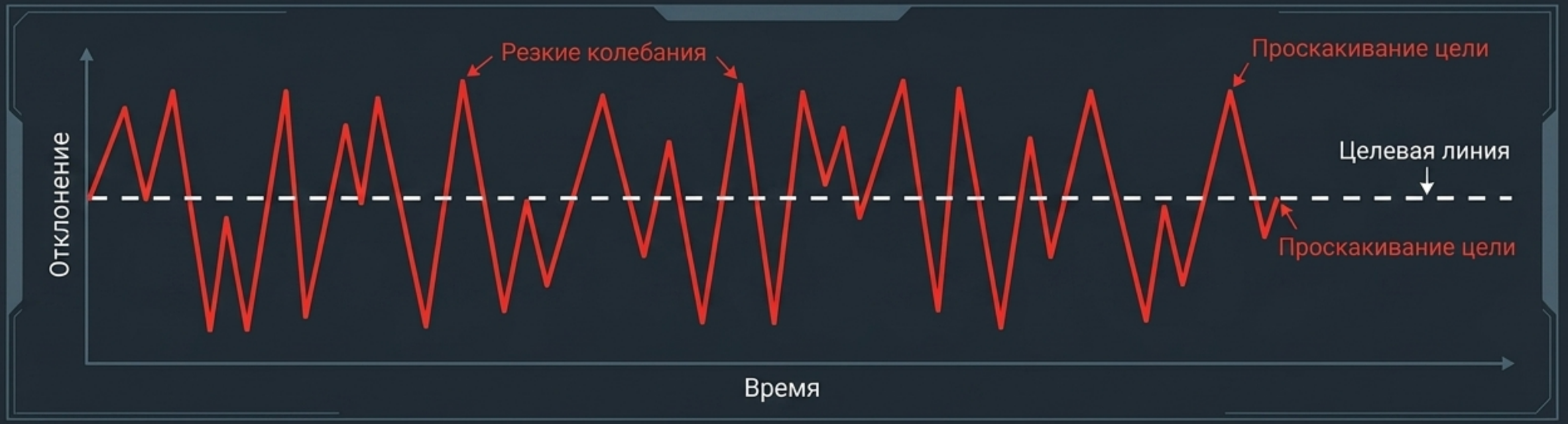
Матрица решений для движения по линии

Левый датчик	Правый датчик	Действие
Белое	Белое	Искать линию / Ехать прямо
Чёрное	Белое	Корректировать влево
Белое	Чёрное	Корректировать вправо
Чёрное	Чёрное	Перекрёсток / Остановка

Набор Keyestudio KS0398 не имеет мобильной платформы, но эта логика является стандартом для автономной навигации по траектории.

NotebookLM

Проблема резких алгоритмов (Bang-Bang Control)



1

Причина:

Простая логика ВКЛ/ВЫКЛ (Если линия слева -> резко вправо).

2

Следствие:

Рывки, потеря точности, износ механизмов, проскакивание цели.

3

Вопрос:

Как научить робота корректировать курс плавно, в зависимости от того, насколько сильно он отклонился?

NotebookLM

Концепция 3: Плавность и PID-регулирование

P (Пропорциональность):
Размер ошибки. Чем дальше от цели, тем сильнее рулим обратно.

I (Интеграл):
Накопленная ошибка. Если нас долго сносит ветром, добавляем постоянную поправку.

D (Производная):
Скорость изменения. Притормаживаем заранее, когда быстро приближаемся к цели.

Ошибка регулирования = Желаемое значение - Текущее значение.

NotebookLM

Золотое правило отладки алгоритмов

Алгоритм почти никогда не получается идеальным с первого раза.

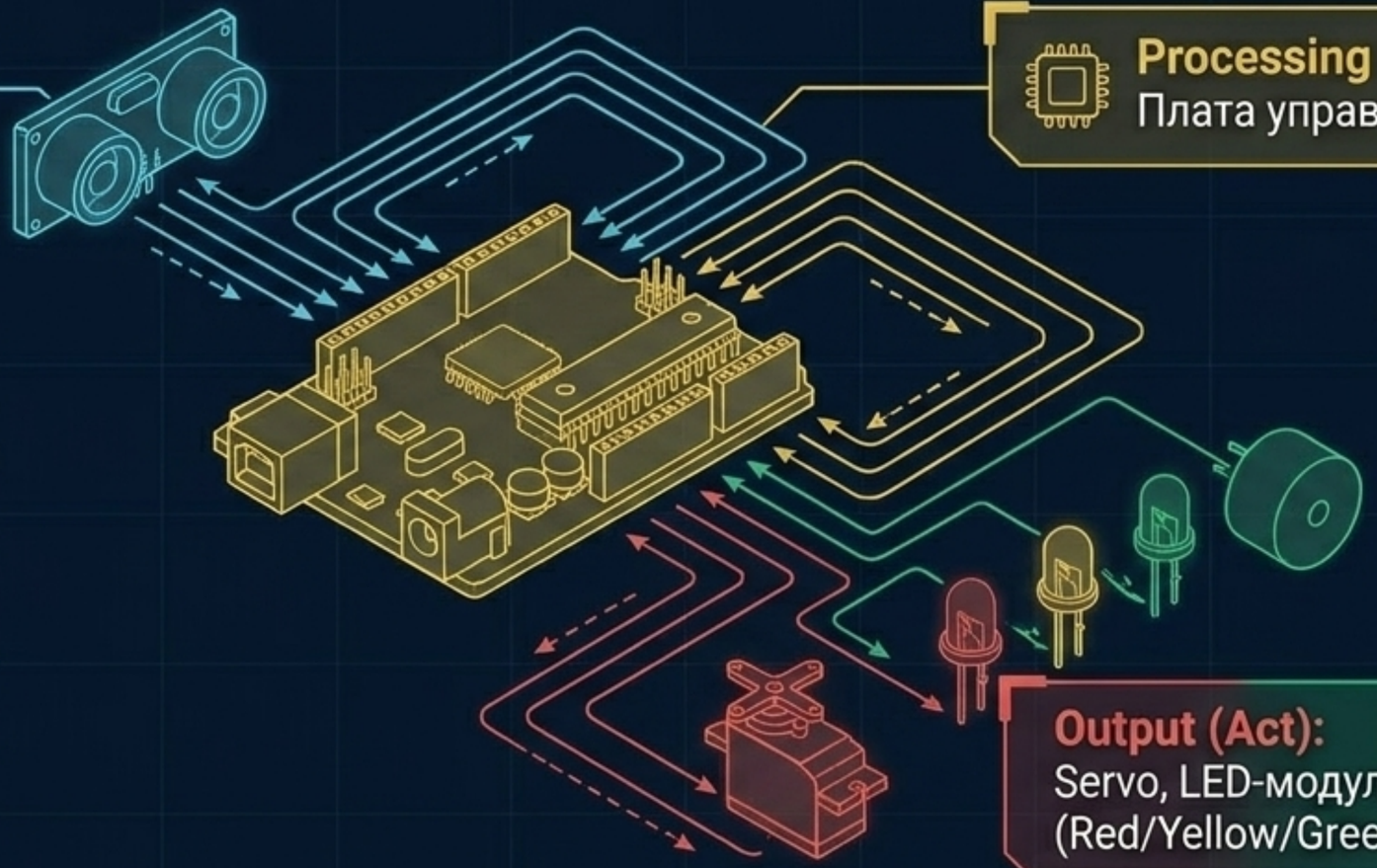
Sensor + Monitor + Logic + LED + Servo → Unified Assembly

1. Проверить датчик отдельно.
2. Вывести сырые значения в монитор порта.
4. Проверить исполнительные модули по одному.
5. Только в конце — объединить всё в единый цикл.

NotebookLM

Практика: Сборка настольного интеллектуального робота

Input (Sense):
Ультразвуковой датчик (или IR)



Processing (Think):
Плата управления

Output (Act):
Servo, LED-модули (Red/Yellow/Green), Buzzer

Наша цель: собрать систему, которая **измеряет расстояние**, определяет **опасную зону**, смотрит по сторонам и выбирает свободное направление.

NotebookLM

Шаг 1: SENSE (Извлечение данных об окружении)

```
int getDistanceCm() {
  // Генерация импульса (trigPin)
  // Замер эха (echoPin)
  int distanceCm = duration * 0.034 / 2;
  return distanceCm;
}
```

Вынос **измерения расстояния в отдельную функцию** — это первый шаг к чистому алгоритму.

Главный цикл программы не должен засоряться микросекундами и пинами.

NotebookLM

Шаг 2: THINK (Маршрутизация данных по зонам)

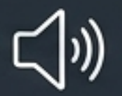
```
int distance = getDistanceCm();
if (distance > 30) {
  // Состояние SAFE
} else if (distance >= 15) {
  // Состояние WARNING
} else {
  // Состояние STOP
}
```

Данные сами по себе бесполезны. Алгоритм **if/else if/else** классифицирует сырые сантиметры в понятные состояния безопасности. (Пороги 30 и 15 см можно калибровать).

NotebookLM

Шаг 3: ACT (Индикация состояний)

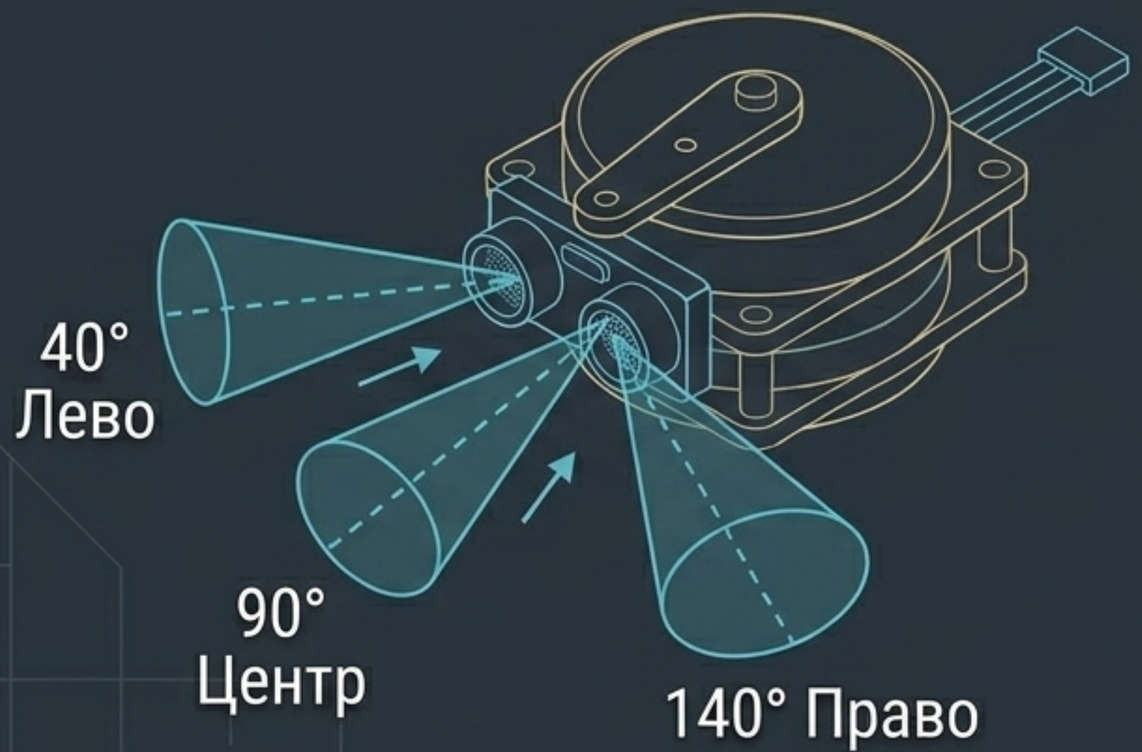
```
void setIndicators(bool green, bool yellow, bool red, bool sound)
```

SAFE	⟶ `setIndicators(true, false, false, false)`
WARNING	⟶ `setIndicators(false, true, false, false)`
STOP	⟶ `setIndicators(false, false, true, true)` 

Единая функция управления индикацией позволяет менять состояние робота одной строкой кода, избегая путаницы с множеством команд digitalWrite.

NotebookLM

Шаг 4: SENSE+ (Сканирование пространства сервоприводом)




```
scannerServo.write(40);
delay(600); // КРИТИЧЕСКИ ВАЖНО! ⚠
int leftDistance = getDistanceCm();
```

Робот — физический объект. Сервопривод не перемещается мгновенно. Если измерить расстояние сразу после команды поворота, датчик прочитает данные на лету и алгоритм примет ложное решение.

NotebookLM

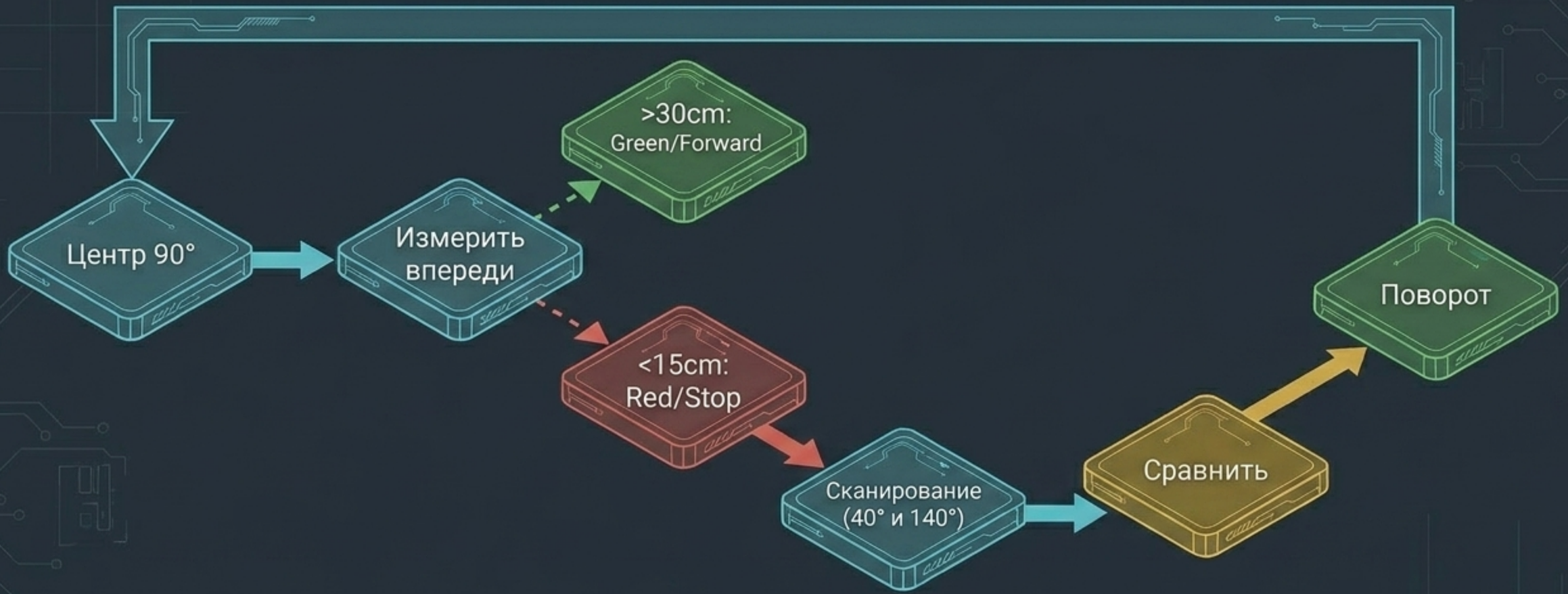
Шаг 5: THINK+ (Сравнение и выбор направления)

```
if (leftDistance < 15 && rightDistance < 15) {
  showStop();
} else if (leftDistance > rightDistance) {
  showTurnLeft();
} else {
  showTurnRight();
}
```

Пик работы алгоритма. Программа сравнивает две переменные, собранные в прошлом шаге, и принимает автономное решение о маршруте.

NotebookLM

Финальная сборка: Архитектура поведения



Алгоритм управления превращает набор электронных компонентов в систему с осмысленным поведением.

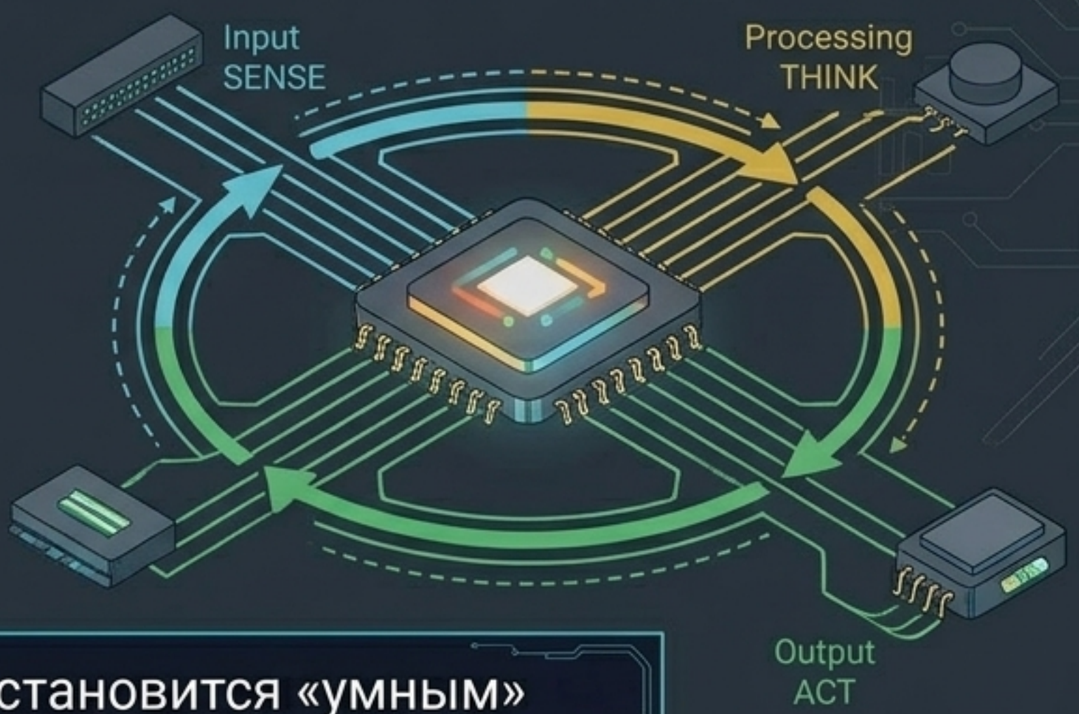
NotebookLM

Диагностика: Типовые ошибки поведения

Симптом	Решение
0 или прыгающие значения	Перепутаны пины или нет паузы Servo. Тест датчика отдельно.
Всегда поворот в одну сторону	Вывести left/right в монитор порта.
Buzzer гудит непрерывно	Явно отключать звук командой noTone.
Горят все LED одновременно	Использовать функцию setIndicators.

NotebookLM

Итоги: От проводов к интеллекту

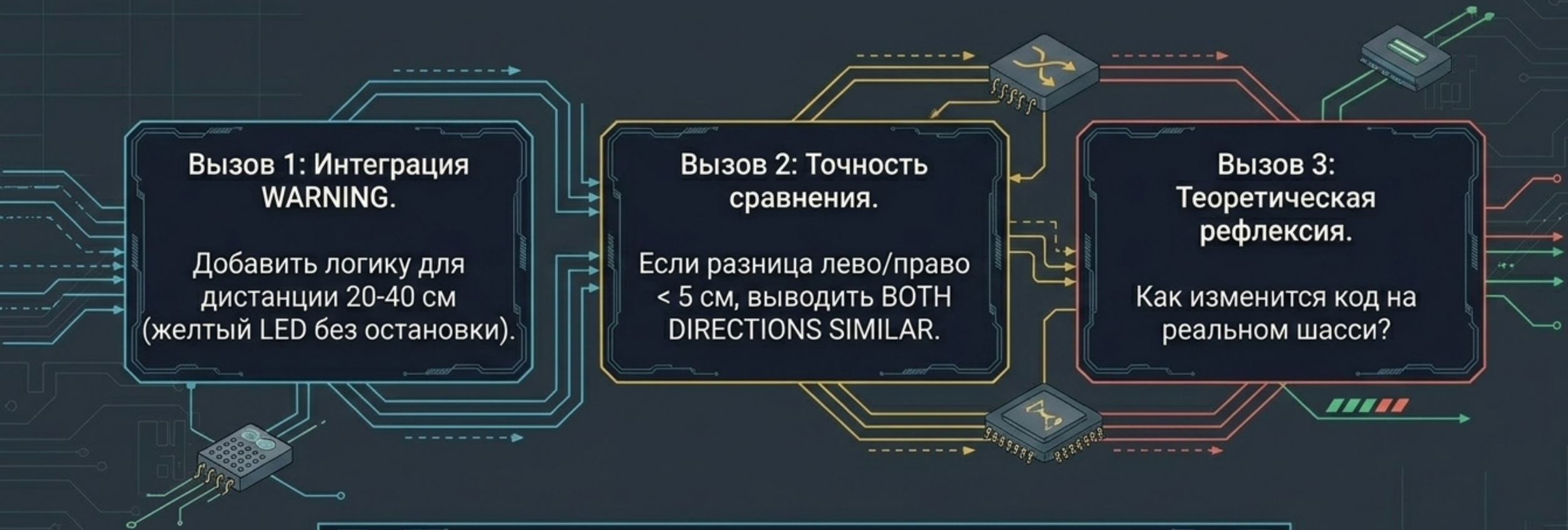


Робототехническое средство становится «умным»
не из-за сложных датчиков или мощных моторов.
Интеллект рождается из алгоритма, который связывает
поток данных с осмысленными действиями.

Сегодня мы перешли от ручного управления к созданию автономной логики.

NotebookLM

Следующий шаг: Усложнение алгоритма



Вызов 1: Интеграция WARNING.

Добавить логику для дистанции 20-40 см (желтый LED без остановки).

Вызов 2: Точность сравнения.

Если разница лево/право < 5 см, выводить BOTH DIRECTIONS SIMILAR.

Вызов 3: Теоретическая рефлексия.

Как изменится код на реальном шасси?

Алгоритмы требуют постоянной оптимизации.
Практикуйтесь в усложнении логических деревьев.

NotebookLM